04 — Data Warehousing with PostgreSQL

Table of Contents

Learning Objectives

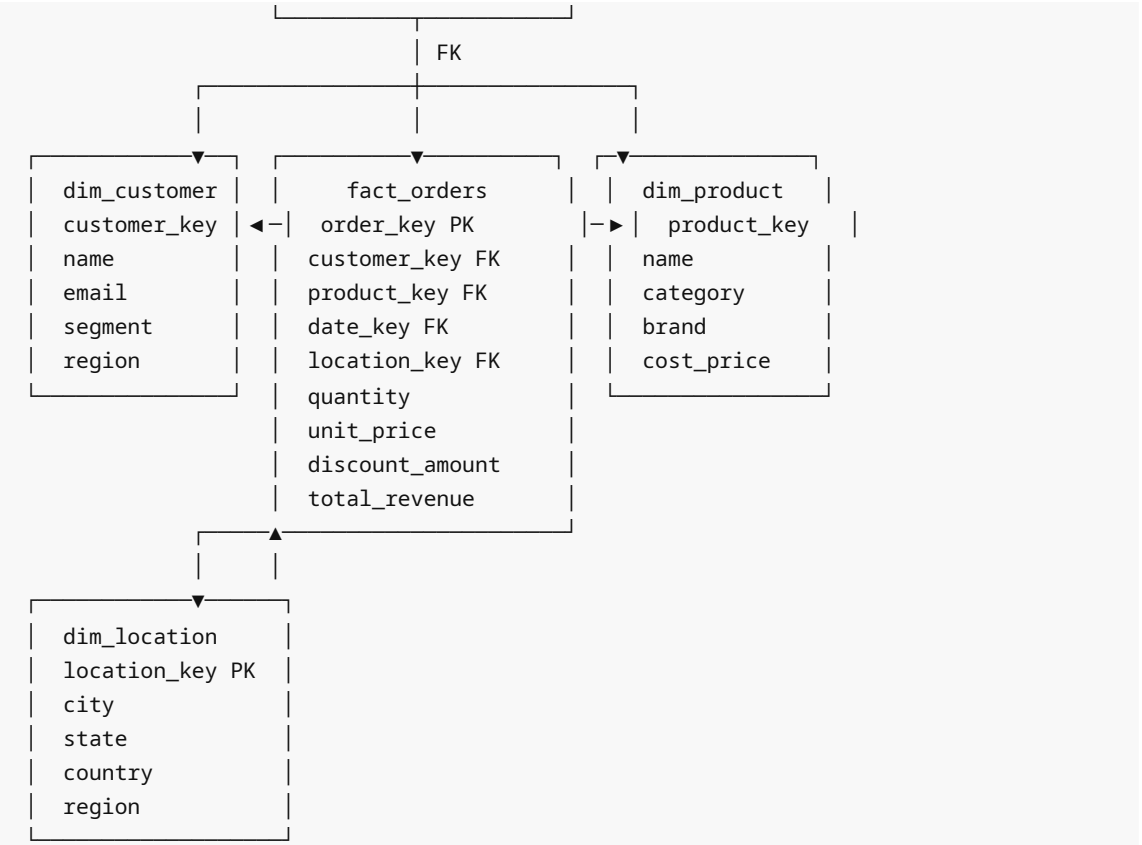
- Design star and snowflake schemas in PostgreSQL
- Build fact and dimension tables with appropriate keys and constraints
- Implement all SCD types (1, 2, 3) in PostgreSQL SQL
- Partition fact tables by date for query performance
- Tune PostgreSQL for analytical / OLAP workloads

OLTP vs OLAP

```
OLTP (Online Transaction Processing)          OLAP (Online Analytical Processing)
──────────────────────────────────────        ──────────────────────────────────────
Short transactions (ms)                       Long scans (seconds–minutes)
Insert/Update heavy                           Read-heavy, aggregation heavy
3NF normalized                                Denormalized (star/snowflake)
Row-oriented storage                          Benefits from columnar storage
Thousands of concurrent users                 Dozens of analysts
pg_stat_activity: many short queries          Few long queries
```

Star Schema

```
                    ┌──────────────────────┐
                    │      dim_date        │
                    │  date_key PK         │
                    │  full_date           │
                    │  year, quarter       │
                    │  month, week, day    │
```



Star Schema DDL

```
-- Date dimension (pre-populated)
CREATE TABLE dim_date (
    date_key        INT         PRIMARY KEY,  -- YYYYMMDD
    full_date       DATE        NOT NULL UNIQUE,
    day_of_week     SMALLINT,
    day_name        VARCHAR(10),
    month_num       SMALLINT,
    month_name      VARCHAR(10),
    quarter         SMALLINT,
    year            SMALLINT,
    is_weekend      BOOLEAN,
    is_holiday      BOOLEAN
);

-- Customer dimension
CREATE TABLE dim_customer (
    customer_key    BIGSERIAL   PRIMARY KEY,
    customer_id     INT         NOT NULL,     -- business key
    name            TEXT        NOT NULL,
    email           TEXT,
    segment         TEXT,
    region          TEXT,
    valid_from      DATE        NOT NULL DEFAULT CURRENT_DATE,
```

```
    valid_to        DATE,
    is_current      BOOLEAN     NOT NULL DEFAULT TRUE
);

-- Product dimension
CREATE TABLE dim_product (
    product_key     BIGSERIAL   PRIMARY KEY,
    product_id      INT         NOT NULL,
    name            TEXT        NOT NULL,
    category        TEXT,
    brand           TEXT,
    cost_price      NUMERIC(10,2),
    valid_from      DATE        NOT NULL DEFAULT CURRENT_DATE,
    valid_to        DATE,
    is_current      BOOLEAN     NOT NULL DEFAULT TRUE
);

-- Location dimension
CREATE TABLE dim_location (
    location_key    BIGSERIAL   PRIMARY KEY,
    city            TEXT,
    state           TEXT,
    country         TEXT        NOT NULL,
    region          TEXT,
    postal_code     TEXT
);

-- Fact table (orders)
CREATE TABLE fact_orders (
    order_key       BIGSERIAL   PRIMARY KEY,
    order_id        BIGINT      NOT NULL,     -- degenerate dimension
    customer_key    BIGINT      NOT NULL REFERENCES dim_customer(customer_key),
    product_key     BIGINT      NOT NULL REFERENCES dim_product(product_key),
    date_key        INT         NOT NULL REFERENCES dim_date(date_key),
    location_key    BIGINT      REFERENCES dim_location(location_key),
    quantity        INT         NOT NULL DEFAULT 1,
    unit_price      NUMERIC(10,2) NOT NULL,
    discount_amount NUMERIC(10,2) DEFAULT 0,
    total_revenue   NUMERIC(12,2) GENERATED ALWAYS AS
                    (quantity * unit_price - discount_amount) STORED
);

-- Indexes on FK columns (critical for star schema joins)
CREATE INDEX idx_fact_orders_customer  ON fact_orders(customer_key);
CREATE INDEX idx_fact_orders_product   ON fact_orders(product_key);
CREATE INDEX idx_fact_orders_date      ON fact_orders(date_key);
CREATE INDEX idx_fact_orders_location  ON fact_orders(location_key);
```

Snowflake Schema

In a snowflake schema, dimensions are further normalized:

```
dim_product ──► dim_category ──► dim_department
dim_customer ──► dim_region ──► dim_country
```

```sql
-- Snowflake: normalize category out of dim_product
CREATE TABLE dim_category (
    category_key    SERIAL  PRIMARY KEY,
    category_name   TEXT    NOT NULL,
    department_key  INT     REFERENCES dim_department(department_key)
);

CREATE TABLE dim_product (
    product_key     BIGSERIAL PRIMARY KEY,
    product_id      INT       NOT NULL,
    name            TEXT      NOT NULL,
    category_key    INT       REFERENCES dim_category(category_key),
    brand           TEXT,
    cost_price      NUMERIC(10,2)
);
```

Star vs Snowflake tradeoff:

- Star: faster queries (fewer joins), more storage, denormalized
- Snowflake: normalized, less storage, slower queries (more joins)
- For PostgreSQL analytics: star schema typically outperforms

---

Fact Tables

Types of Facts

```sql
-- Transactional fact (one row per transaction event)
CREATE TABLE fact_sales (
    sale_key        BIGSERIAL PRIMARY KEY,
    sale_date_key   INT,
    customer_key    BIGINT,
    product_key     BIGINT,
    quantity        INT,
    revenue         NUMERIC(12,2)
);

-- Snapshot fact (state at a point in time)
CREATE TABLE fact_inventory_snapshot (
    snapshot_date_key INT,
    product_key       BIGINT,
    warehouse_key     INT,
    quantity_on_hand  INT,
    reorder_level     INT,
    PRIMARY KEY (snapshot_date_key, product_key, warehouse_key)
);

-- Accumulating snapshot fact (tracks lifecycle)
```

```sql
CREATE TABLE fact_order_lifecycle (
    order_key          BIGINT PRIMARY KEY,
    order_date_key     INT,
    ship_date_key      INT,    -- NULL until shipped
    deliver_date_key   INT,    -- NULL until delivered
    return_date_key    INT,    -- NULL unless returned
    days_to_ship       INT GENERATED ALWAYS AS (ship_date_key - order_date_key)
STORED
);
```

Dimension Tables

Populate Date Dimension

```sql
INSERT INTO dim_date (date_key, full_date, day_of_week, day_name,
                      month_num, month_name, quarter, year, is_weekend)
SELECT
    TO_CHAR(d, 'YYYYMMDD')::INT,
    d,
    EXTRACT(DOW FROM d)::SMALLINT,
    TO_CHAR(d, 'Day'),
    EXTRACT(MONTH FROM d)::SMALLINT,
    TO_CHAR(d, 'Month'),
    EXTRACT(QUARTER FROM d)::SMALLINT,
    EXTRACT(YEAR FROM d)::SMALLINT,
    EXTRACT(DOW FROM d) IN (0, 6)
FROM GENERATE_SERIES(
    '2020-01-01'::DATE,
    '2030-12-31'::DATE,
    '1 day'::INTERVAL
) AS d;
```

Slowly Changing Dimensions (SCDs)

SCD Type 1 — Overwrite (no history)

```sql
UPDATE dim_customer
SET segment = 'premium', email = 'new@example.com'
WHERE customer_id = 1001 AND is_current = TRUE;
```

SCD Type 2 — Add new row (full history)

```sql
-- Expire old record
UPDATE dim_customer
SET is_current = FALSE, valid_to = CURRENT_DATE - 1
WHERE customer_id = 1001 AND is_current = TRUE;
```

```
-- Insert new record
INSERT INTO dim_customer (customer_id, name, email, segment, valid_from, valid_to,
is_current)
SELECT
    customer_id, name, 'updated@example.com', 'premium',
    CURRENT_DATE, NULL, TRUE
FROM dim_customer
WHERE customer_id = 1001 AND is_current = FALSE
ORDER BY valid_to DESC
LIMIT 1;
```

SCD Type 2 — Efficient MERGE pattern

```
-- Full SCD Type 2 upsert using a single procedure
CREATE OR REPLACE PROCEDURE scd2_upsert_customers(
    p_customer_id INT,
    p_name TEXT,
    p_email TEXT,
    p_segment TEXT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_changed BOOLEAN;
BEGIN
    -- Check if anything changed
    SELECT (email <> p_email OR segment <> p_segment)
    INTO v_changed
    FROM dim_customer
    WHERE customer_id = p_customer_id AND is_current = TRUE;

    IF NOT FOUND THEN
        -- New customer
        INSERT INTO dim_customer (customer_id, name, email, segment)
        VALUES (p_customer_id, p_name, p_email, p_segment);
    ELSIF v_changed THEN
        -- Expire old
        UPDATE dim_customer
        SET is_current = FALSE, valid_to = CURRENT_DATE
        WHERE customer_id = p_customer_id AND is_current = TRUE;

        -- Insert new
        INSERT INTO dim_customer (customer_id, name, email, segment, valid_from)
        VALUES (p_customer_id, p_name, p_email, p_segment, CURRENT_DATE);
    END IF;
    -- If not changed: do nothing (SCD Type 1 for name)
END;
$$;
```

SCD Type 3 — Add previous value column

```sql
-- SCD Type 3: track only current and previous value
ALTER TABLE dim_customer ADD COLUMN prev_segment TEXT;
ALTER TABLE dim_customer ADD COLUMN segment_changed_at DATE;

UPDATE dim_customer
SET prev_segment       = segment,
    segment            = 'premium',
    segment_changed_at = CURRENT_DATE
WHERE customer_id = 1001;
```

Partitioning for Warehousing

```sql
-- Fact table partitioned by month
CREATE TABLE fact_orders (
    order_key       BIGSERIAL,
    order_date      DATE        NOT NULL,
    customer_key    BIGINT,
    product_key     BIGINT,
    total_revenue   NUMERIC(12,2),
    PRIMARY KEY (order_key, order_date)
) PARTITION BY RANGE (order_date);

-- Create monthly partitions
CREATE TABLE fact_orders_2024_01
    PARTITION OF fact_orders
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

CREATE TABLE fact_orders_2024_02
    PARTITION OF fact_orders
    FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');

-- Auto-create partitions for future months (pg_partman extension)
-- SELECT partman.create_parent('public.fact_orders', 'order_date', 'range',
'monthly');
```

Columnar Storage

PostgreSQL is row-oriented by default. For heavy analytics:

```sql
-- Option 1: Use Citus columnar (open source)
-- CREATE TABLE fact_orders (...) USING columnar;

-- Option 2: pg_analytics / DuckDB FDW for columnar scans

-- Option 3: Materialized views + partial indexes approximate columnar benefits
CREATE MATERIALIZED VIEW mv_monthly_revenue AS
SELECT
    DATE_TRUNC('month', o.order_date)::DATE AS month,
```

```
    p.category,
    l.country,
    SUM(o.total_revenue)    AS revenue,
    COUNT(*)                AS order_count
FROM fact_orders o
JOIN dim_product  p USING (product_key)
JOIN dim_location l USING (location_key)
GROUP BY 1, 2, 3;

CREATE UNIQUE INDEX ON mv_monthly_revenue (month, category, country);
```

Common Mistakes

Mistake	Impact	Fix
No surrogate keys in dimensions	Joins on business keys are slow	Always use surrogate keys (BIGSERIAL)
Storing facts without date dimension	Cannot do date-based analysis	Always include date_key FK
No indexes on FK columns in fact table	Slow star-schema joins	Index every FK column in fact table
SCD Type 2 without `is_current` flag	Complex queries to find current record	Always add `is_current BOOLEAN`
Updating facts (facts are immutable)	Breaks analytical consistency	Append new fact row, mark old as reversed
Missing ANALYZE after bulk DIM load	Bad cardinality estimates	ANALYZE dim_* after every load

---

Best Practices

1. **Surrogate keys** in all dimension tables — never join on business keys.
2. **Date dimension** populated for the full date range before any fact loads.
3. **Immutable facts** — facts are insert-only; corrections use reversal rows.
4. **Partition fact tables** by date (monthly) for efficient range scans.
5. **Partial indexes** for current dimension records: `WHERE is_current = TRUE`.
6. **BRIN indexes** on monotonically increasing date columns in fact tables.
7. `CLUSTER` fact table on date_key periodically for sequential scan speed.

Performance Considerations

```
-- BRIN index for large time-series fact tables
CREATE INDEX idx_fact_orders_date_brin ON fact_orders USING BRIN (order_date)
    WITH (pages_per_range = 128);
```

```sql
-- Partial index for current dimension rows
CREATE INDEX idx_dim_customer_current
    ON dim_customer (customer_id)
    WHERE is_current = TRUE;

-- Enable parallel query for analytical workloads
SET max_parallel_workers_per_gather = 8;
SET enable_parallel_hash = ON;

-- Typical star schema analytical query
EXPLAIN ANALYZE
SELECT
    d.year,
    d.quarter,
    c.region,
    p.category,
    SUM(f.total_revenue)    AS revenue,
    COUNT(DISTINCT f.customer_key) AS unique_customers
FROM fact_orders f
JOIN dim_date     d ON d.date_key     = f.date_key
JOIN dim_customer c ON c.customer_key = f.customer_key AND c.is_current = TRUE
JOIN dim_product  p ON p.product_key  = f.product_key  AND p.is_current = TRUE
WHERE d.year = 2024
GROUP BY 1, 2, 3, 4
ORDER BY 1, 2, 3, revenue DESC;
```

Interview Questions & Answers

Q1: What is the difference between a star schema and a snowflake schema?

> *Star: dimension tables are flat (denormalized), connected directly to the fact table — faster queries, more storage. Snowflake: dimension tables are normalized into sub-dimensions — less storage, more joins, slower queries.*

Q2: What is a surrogate key and why use it in a data warehouse?

> *A surrogate key is a system-generated integer (BIGSERIAL) with no business meaning. It insulates the warehouse from source system key changes, enables SCD Type 2 (multiple rows per business key), and provides fast integer joins.*

Q3: Explain SCD Type 2 and how you implement it.

> *SCD Type 2 preserves full history by adding a new row on change. Implementation: UPDATE old row (`is_current=FALSE`, `valid_to=today`), INSERT new row (`is_current=TRUE`, `valid_from=today`). Query current state with `WHERE is_current = TRUE`.*

Q4: What is a degenerate dimension?

> *A fact attribute that would normally be a dimension but has no dimension table — for example, `order_id` stored in the fact table. It carries context but doesn't warrant its own table.*

Q5: How do you handle late-arriving facts?

> *Insert with the correct historical `date_key`. If aggregates were pre-computed, mark them as stale and re-aggregate. Materialized views can be refreshed selectively.*

Q6: Why are facts considered immutable?

> *Once recorded, a fact (sale, click, transaction) is history and shouldn't change. Corrections are handled by inserting a reversal row (negative quantities) and a new corrected row, preserving the audit trail.*

Q7: How do you populate a date dimension in PostgreSQL?

> *Use `GENERATE_SERIES` from start date to end date, extracting `year`, `month`, `quarter`, `day_of_week` etc. with `EXTRACT` and `TO_CHAR`. Load once and it never changes.*

Q8: What is a BRIN index and when is it useful for warehousing?

> *BRIN (Block Range Index) stores min/max values per block range. Extremely small, perfect for monotonically increasing columns like `order_date` in fact tables where data is inserted in order.*

Q9: How do you optimize a star-schema query in PostgreSQL?

> *(1) Index all FK columns in fact table, (2) Use partial indexes for `is_current=TRUE` in dims, (3) Enable parallel query, (4) CLUSTER fact table on date_key, (5) Use materialized views for frequently queried aggregates.*

Q10: What is an accumulating snapshot fact table?

> *A fact table with multiple date FKs that are updated as a business process advances through stages (order_date, ship_date, deliver_date). Unlike transactional facts, these rows are updated in place.*

Exercises & Solutions

Exercise 1: Build a mini star schema for a retail store

```
-- Dimension: store
CREATE TABLE dim_store (
    store_key   SERIAL  PRIMARY KEY,
    store_id    INT     NOT NULL,
    store_name  TEXT,
    city        TEXT,
    state       TEXT
);

-- Fact: daily sales
CREATE TABLE fact_daily_sales (
    sale_key    BIGSERIAL PRIMARY KEY,
    date_key    INT    REFERENCES dim_date(date_key),
    store_key   INT    REFERENCES dim_store(store_key),
    product_key BIGINT REFERENCES dim_product(product_key),
    quantity    INT,
    revenue     NUMERIC(12,2)
);
```

Exercise 2: SCD Type 2 for product price changes

```sql
-- Record a price change for product 501
UPDATE dim_product SET is_current=FALSE, valid_to=CURRENT_DATE - 1
WHERE product_id = 501 AND is_current = TRUE;

INSERT INTO dim_product (product_id, name, category, cost_price, valid_from,
is_current)
SELECT product_id, name, category, 149.99, CURRENT_DATE, TRUE
FROM dim_product WHERE product_id = 501 ORDER BY valid_to DESC LIMIT 1;
```

Real-World Scenarios

Scenario: Retail analytics warehouse A chain with 500 stores loads daily sales files into PostgreSQL. Star schema: fact_sales partitioned monthly, dim_store, dim_product (SCD2 for price/category changes), dim_date. Analysts query rolling 12-month revenue by region, category, and store tier in under 5 seconds.

Scenario: SaaS subscription analytics Monthly subscription snapshots stored in `fact_subscription_snapshot`. Cohort analysis: group by `customer_key` first subscription month and track retention month-over-month using `dim_date` quarter/month fields.

Cross-References

- `02_elt_patterns.md` — SCD transforms in ELT pipelines
- `07_analytics_workloads.md` — window functions on top of star schema
- `05_batch_processing.md` — bulk loading fact tables efficiently
- `09_postgresql_and_dbt.md` — dbt for DW model management